Mini Nmap

Project Overview

Mini Nmap is a python based project meant to showcase basic cybersecurity and networking skills and concepts. Designed to be a scanning tool this project aims to replicate Nmap's functionality whilst highlighting the practical skills of working with python socket library.

Purpose

- Deepen understanding of TCP/IP networking.
- Gain experience Python socket programming.
- Learn how network scanning tools work.
- Develop practical cybersecurity skills understanding.

Scope

Included

- TCP port scanning.
- UDP port scanning.
- Custom port range selection.
- Scan result reporting.

Not Included

- Vulnerability scanning.
- Scripting.
- Stealth scanning techniques.
- Packet Crafting.
- OS fingerprinting.
- Service Identification.
- Host reachability checks.

Constraints

Constraint	Description
Educational Purpose	The project is intended for learning and demonstrating cybersecurity concepts.
Manual Implementation	Core scanning functionality must be implemented using Python sockets rather than third-party scanning tools.
Command Line Interface	The application will operate through a text based interface.

Functional Requirements

Requirement	Explanation	Complexity
Target Host Input	The application should allow the user to select a target IP address.	Low
Port Range Selection	The application should allow the user to set a custom range of ports to scan.	Low
TCP Port Scanning	The application should identify open TCP ports on the target host.	Medium
UDP Port Scanning	The application should identify open UDP ports on the target host.	High
Scan Reporting	The application should display scan results in a clear and structured format.	Medium
Error Handling	The application should handle invalid inputs, unreachable hosts, and network timeouts without crashing.	Medium

Non-Functional Requirements

Requirement	Explanation
Performance	Scans should be efficient and not hang indefinitely.
Reliability	The application should continue scanning even if individual port fails.
Usability	Application should provide clear instructions and needs to be easy to read.

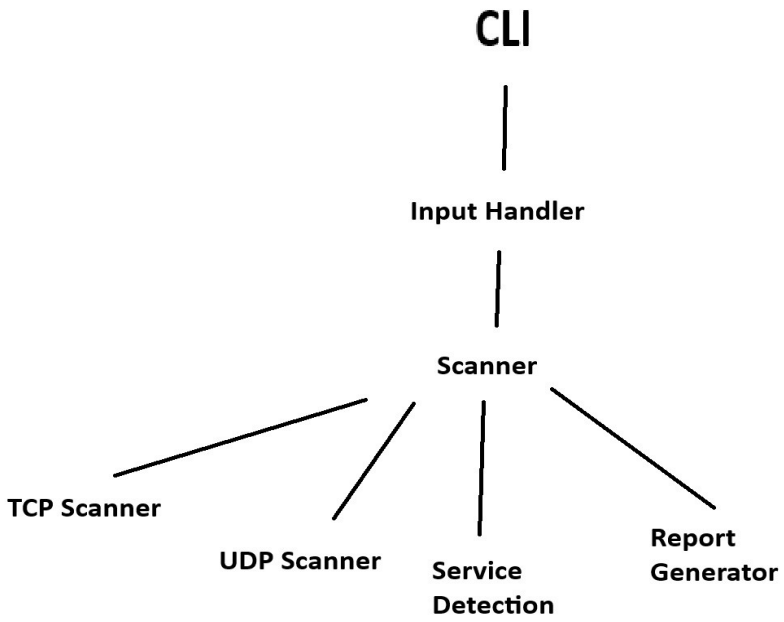
Technology Stack

Component	Technology
Programming Language	Python
Networking Library	Socket
Development Environment	Visual Studio Code
Repository	GitHub

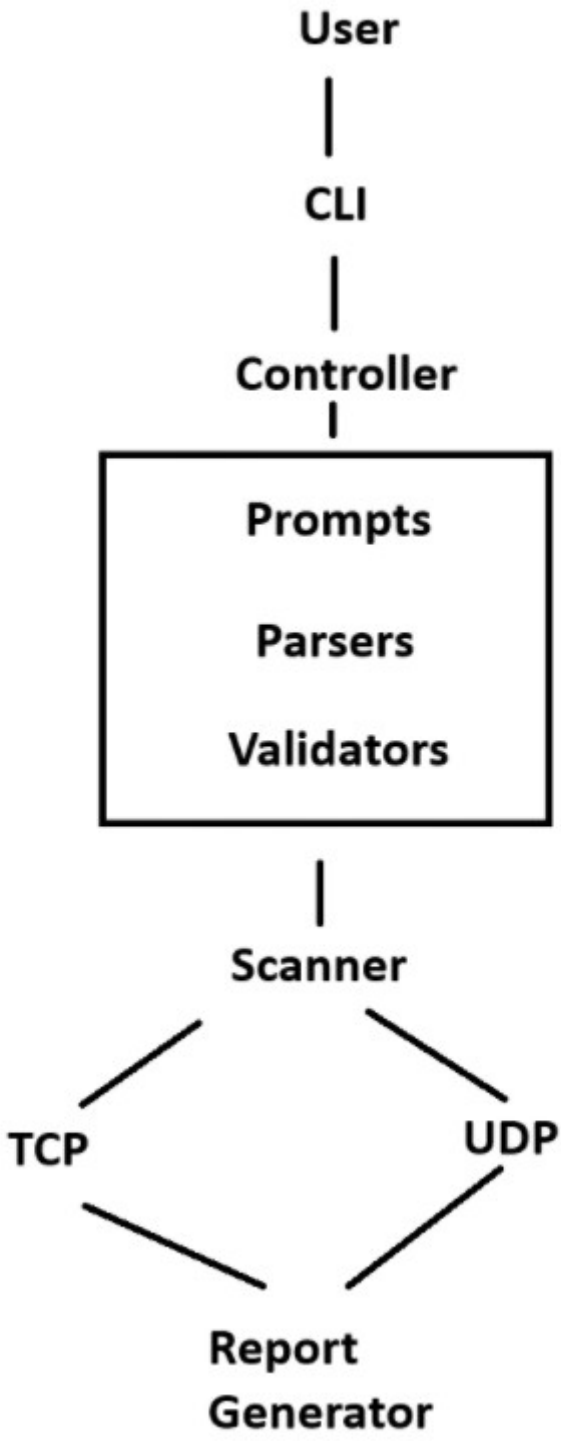
Testing Environment	VirtualBox Virtual Machines
Operating Systems	Windows & Linux

Architecture

Initial Design:



Improved Design:

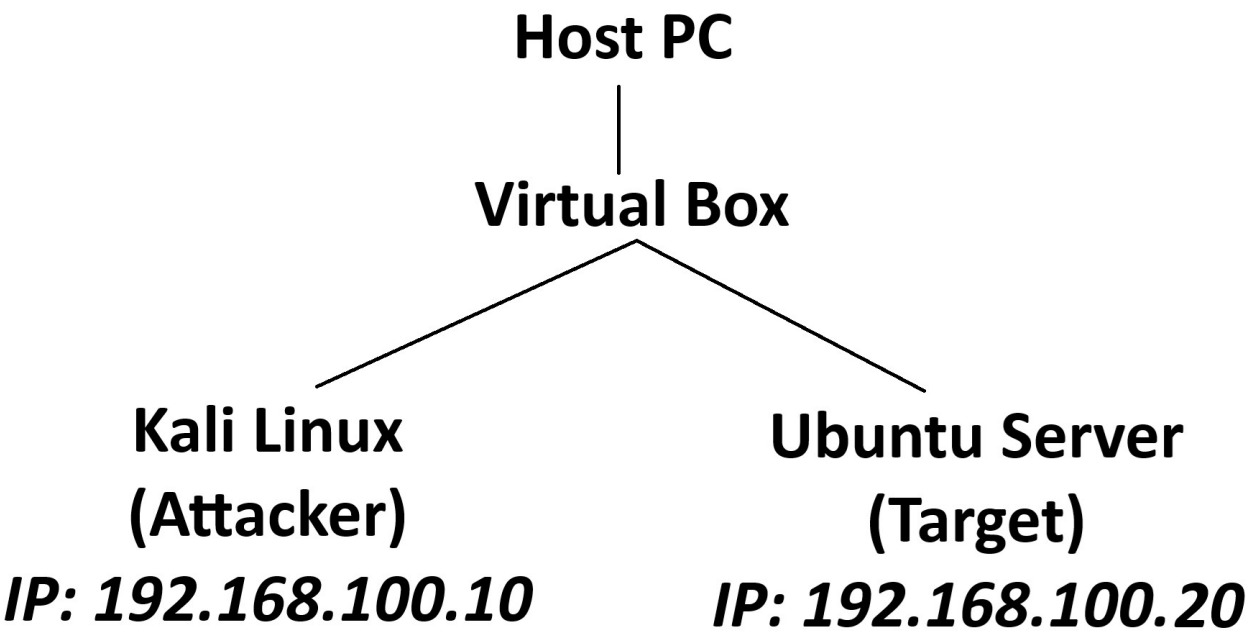


Platform Support

The application is intended to operate on both Windows and Linux systems.

- Python standard libraries will throughout the whole project.
- OS specific functionality will be not be used

- Socket operations will be implemented using Pythons built in networking libraries.



Security & Legal Considerations

Mini Nmap is developed for educational and authorised security testing purposes only.

The following measures are intended to help ensure responsible use:

- Scanning should only be performed against systems where explicit permission has been granted.
- The application will not include features designed to bypass security controls.
- No stealth or evasion techniques will be implemented in the initial version.
- Scan results will be stored locally and not transmitted to external services.
- Testing will be conducted within controlled lab environments using virtual machines.
- The project is intended to demonstrate networking and cybersecurity concepts rather than facilitate unauthorised access.

Authorised Use Only: This tool is intended for educational purposes and authorised network testing. Users are responsible for ensuring they have permission before scanning any target system.

Success Criteria

The project will be considered successful if:

- Users can specify a target host using IP address.
- Users can scan custom port ranges.
- Open TCP ports are correctly identified.

- Open UDP ports are correctly identified.
- Results are displayed clearly.
- The application handles errors without crashing.
- Scan results closely match equivalent Nmap scans.

Testing

Testing Strategy

This testing strategy will cover input validation, CLI and scanning components of Mini Nmap.

Unit Testing

Individual functions will be tested to ensure correct operation. Unit tests will be constructed using the unit test library in python in accordance with Arrange Act Assert (AAA) pattern structure. Utilising the project's design documentation the most applicable section to perform unit testing will be the validation process.

The code used for validating and parsing inputs will be the most isolated part of the code. Due to no interactions with any external parts of the program such as the network components the ideal scenario for unit testing is presented and is what will be focused on during the tests.

Functions interacting with the CLI or network components will be tested using the unittest.mock to isolate behaviour.

Below is an example of unit testing format:

Test	Arrange	Assert
1	timeout=0	valid=False

Comparative Testing

By cross referencing the different outputs of the program and comparing with Nmap results allows bugs and problems with the code can be detected easily. As Nmap is a well documented and widely trusted program any deviating Mini Nmap outcome will be treated as wrong/incorrect. Through the use of different simulated scenarios Mini Nmap and Nmap can be directly compared on how they behave towards different inputs and if the outputs match.

Below is an example of comparative testing format:

Test	Protocol	Port	Scenario	Nmap Result	Mini Nmap Result	Outcome
1	TCP	80	Service stopped	Open	Closed	FAIL
2	UDP	53	UDP Scan	Open	Open	PASS

Error Testing

Error testing focuses on unexpected inputs and error cases. Using these tests will help to ensure the program is responding to errors correctly and behaves predictably.

As error handling occurs at the CLI level in this program the best way to design error tests will be by looking at the functional requirements and test how the program reacts from the user's perspective.

Below is an example of error testing format:

Test	Input Field	Invalid Input	Expected Behaviour	Observed Behaviour	Outcome
1	Target IP	0.0.0.0	User will be shown error and asked to retype the IP.	Error was shown and user asked to retype IP address.	PASS
2	Port Range	0 (min)	Error "Incorrect Port Range" will be shown as port is below minimum threshold.	"Incorrect Port Range" Error shown.	PASS
3	Port Range	1(min) and 65535(max)	Program will work as expected even though boundary values are present.	Program crashed due to an error.	FAIL

Test Results

Unit Testing

Test Date: 12/08/2026

Test Objective: Unit testing will be used to evaluate the initial validation functions of the program. Individual functions are tested using Python's unit test library following the Arrange-Act-Assert (AAA) pattern.

Testing focuses on:

Valid Inputs – Valid inputs that produce expected results.

Boundary Values – Minimum/maximum values that are on the edge.

Invalid Inputs – Out of range or incorrect values.

Test Location:

All the unit tests can be found here.

https://github.com/EKuszynski04/mini-Nmap/blob/main/tests/unittesting.py

validate_ports

Test	Arrange	Assert	Type

1	min=1, max=100	valid=True	Valid Inputs
2	min=80, max=80	valid=True	Boundary (single port)
3	min=100, max=1	valid=False	Invalid (min bigger than max)
4	min=0, max=100	valid=False	Invalid (below min)
5	min=1, max=MAX_PORT_RANGE	valid=True	Valid (max port range)
6	min=1, max=MAX_PORT_NUMBER	valid=True	Boundary (maximum port number)
7	min=1, max=MAX_PORT_NUMBER+1	valid=False	Boundary (over max number)
8	min=-1, max=100	valid=False	Invalid(negative number)

validate_timeout

Test	Arrange	Assert	Type
1	timeout=MIN_TIMEOUT_LIMIT	valid=True	Boundary (min timeout)
2	timeout=MIN_TIMEOUT_LIMIT - 1	valid=False	Invalid (below min)
3	timeout=MAX_TIMEOUT_LIMIT	valid=True	Boundary (max timeout)
4	timeout=MAX_TIMEOUT_LIMIT + 1	valid=False	Invalid (above max)
5	Timeout=0	valid=False	Invalid (zero)
6	Timeout=-1	valid=False	Invalid (negative)
7	Timeout=1.111	valid=True	Valid (float number)

validate_scan_type

Test	Arrange	Assert	Type
1	TCP	valid=True	Valid Inputs (upper case)
2	udp	valid=True	Valid Inputs (lower case)
3	tCP	valid=True	Valid Inputs (mix case)
4	“”	valid=False	Invalid (empty input)
5	123	valid=False	Invalid (int input)

Unit Testing – Update

As the project continued to be development new tests had to be conducted to accommodate for the changes in design and code structure. In accordance to the new architecture design new unit tests had to be conducted on the controller class that now orchestrates the validation and parses classes. The controller class contains code that decides the flow of the application, dispatch logic and data transformations. Due to its importance and as these behaviours cannot be tested only using validators and parser a separate unit tests were designed.

Parsers are intentionally not unit tested. This is due to parsers being thinly wrapped in Python's built-in int() and float() standard library wrappers.

ScanController

using unittest.mock.patch

Test	Arrange	Assert	Type
1	Call TCP function	run_scan calls tcp_scan	Valid (scan type valid)
2	Call UDP function	run_scan calls udp_scan	Valid (scan type valid)
3	Mock get_target_ip with valid IP	Returns a valid and parsed IP.	Valid (valid IP string)
4	Mock get_target_ip with invalid IP	ERROR_IP_INVALID printed and loop restarts	Invalid (invalid IP string)
5	Mock get_ports min=100 max=1	ERROR_PORT_RANGE printed and loop restarts until valid ports inputted	Invalid (min > max)
6	Mock get_ports min=1 max=20	Returns list of ports	Valid (min < max)
7	Mock get_timeout = [0,1]	ERROR_MIN_TIMEOUT printed and loop restarts until valid timeout inputted	Invalid (zero)
8	get_timeout = [100,1]	ERROR_MAX_TIMEOUT printed and loop restarts until valid timeout inputted	Invalid (above max)
9	All prompts valid	Returns ScanConfig and loop closes	Valid (all prompts valid)

Comparative Testing

Test Date: 15/08/2026

Test Objective: Test is designed to evaluate the initial program and to highlight any potential problems with code by comparing with Nmap. Main goal is to identify how Mini Nmap and official Nmap differ in practice.

Test Location:

All the tests can be found here.

https://github.com/EKuszynski04/mini-Nmap/blob/main/tests/functional_test.txt

Test Results Summary:

Test	Protocol	Port	Scenario	Nmap Result	Mini Nmap Result	Outcome
1	TCP	22,80,443, 445,3306	Mixed port states	Open/ Closed	Open/ Closed	PASS
2	TCP	80	Service stopped	Closed	Closed	PASS
3	TCP	9999	Firewall filtered	Filtered	Error	FAIL
4	UDP	53	UDP scan	Open/ Filtered	Open	PARTIAL
5	UDP	22222	No known UDP service	Open/ Filtered	Open	PARTIAL

Test 1

Objective: Verify that Mini Nmap basic TCP port scanning works and can distinguish between open and closed ports.

Result: Pass. Mini Nmap and Nmap has matching results.

Test 2

Objective: Verify if Mini Nmap correctly changes ports status after a service has been stopped.

Configuration: HTTP service apache was stopped on the ubuntu machine.

Result: Pass. Mini Nmap and Nmap has matching results.

Test 3

Objective: Verify if Mini Nmap distinguishes if a port has been filtered by firewall.

Configuration: Ubuntu firewall was configured to filter traffic to port 9999.

Result: Fail. Mini Nmap displayed an error status instead of a timeout state.

Proposed Improvements: Investigate the TCP function and fix how it handles timeouts. Add a filtered state. Perform unit tests on function to see if It works.

Test 4

Objective: Verify if Mini Nmap UDP scanning works on ports with UDP services running.

Configuration: Scan port 53 using UDP.

Result: Partial. Mini Nmap displayed open whereas Nmap displayed an open|filtered. As there was no UDP response test was not indicative of a port being opened.

Proposed Improvements: Write code to send UDP probes and test responses whilst comparing with Nmap once variables such as the response can be controlled.

Test 5

Objective: Verify if Mini Nmap UDP scanning works on ports that have no UDP service running.

Configuration: Scan port 22222 using UDP.

Result: Partial. Mini Nmap displayed open whereas Nmap displayed an open|filtered.

Proposed Improvements: Analyse how current UDP function code works and redesign using UDP responses instead of relying on connect_ex().

Main Takeaways: TCP works well on basic ports however needs better classification when dealing with firewalls and timeouts. UDP code needs to be rewritten as current code doesn't work correctly when compared to Nmap. A better testing environment also needs to be developed for UDP specifically to do with server side responses.

Comparative Testing – Part 2

Test Date: 04/09/2026

Test Objective: Test is designed to evaluate updated UDP code by comparing Mini Nmap new code results with Nmap.

Test Location:

All the test results can be found here.
https://github.com/EKuszynski04/mini-Nmap/blob/main/tests/functional_test.txt

Test Results Summary:

Test	Protocol	Port	Scenario	Nmap Result	Mini Nmap Result	Outcome
1	UDP	9999	Open port with UDP server	Open	Open	PASS
2	UDP	9998	No UDP service	Open| filtered	Timeout	PASS
3	UDP	9997	Closed UDP port	Closed	Closed	PASS

Test 1

Objective: Verify that Mini Nmap UDP port scanning works and can recognise open ports with UDP services running.

Result: Pass. Mini Nmap and Nmap has matching results.

Test 2

Objective: Verify how Mini Nmap reacts to open ports with no UDP service running.

Configuration: UDP server was stopped on the ubuntu machine.

Result: Pass. Mini Nmap and Nmap had differing results however this is due how Mini Nmap displays timeout messages. Mini Nmap uses the state Timeout as opposed to Open|filtered how it is in Nmap and is caused by Mini Nmaps having no connection with the UDP service whilst Nmap assumes its been filtered.

Test 3

Objective: Verify if Mini Nmap recognises open ports with UDP services that are closed.

Configuration: Ubuntu firewall was configured to allow traffic to port 9997/udp.

Result: Pass. Mini Nmap and Nmap has matching results.

Main Takeaways: UDP works on a basic level and matches Nmap's responses. Due to the changes in the code a new socket is created for each port to account for closed UDP services and their ICMP responses. Although there is variation in how the results are displayed the main logic flow remains consistent in how both programs handle various UDP related events at the most basic level.

Error Testing

Test Date: 04/10/2026

Test Objective: Verify that Mini Nmap handles invalid user inputs and the user is guided back to a valid input.

Test	Input Field	Invalid Input	Expected Behaviour	Observed Behaviour	Outcome
1	Target IP	999.999.999.999	ERROR_IP_INVALID shown and then user is re prompted.	"Invalid IP address" message shown and re prompted to enter IP address	PASS
2	Target IP	abc	ERROR_IP_INVALID shown and then user is re prompted.	"Invalid IP address" message shown and re prompted to enter IP address	PASS
3	Target IP	Empty	ERROR_IP_INVALID shown and then user is re prompted.	"Invalid IP address" message shown and re prompted to enter IP	PASS

				address	
4	Port	0	ERROR_PORT_INVALID shown and then user is re prompted.	“Invalid port number” message shown and re prompted to enter port number.	PASS
5	Port	999999	ERROR_PORT_INVALID shown and then user is re prompted.	“Invalid port number” message shown and re prompted to enter port number.	PASS
6	Port	Min=100 Max=1	ERROR_PORT_RANGE shown and then user is re prompted.	“Max port number is smaller than minimum port number.” message shown and re prompted to enter port number.	PASS
7	Port	abc	ERROR_PORT_INVALID shown and then user is re prompted.	“Invalid port number” message shown and re prompted to enter port number.	PASS
8	Scan Type	ICMP	ERROR_SCAN_TYPE is shown and then user is re prompted.	“Invalid scan type” message shown and re prompted to enter scan type.	PASS
9	Timeout	0	ERROR_MIN_TIMEOUT is shown and then user is re prompted.	“Invalid timeout value. Minimum timeout exceeded” message shown and re prompted to enter timeout value.	PASS
10	Timeout	100	ERROR_MAX_TIMEOUT is shown and then user is re prompted.	“Invalid timeout value. Maximum timeout exceeded” message shown and re prompted to enter timeout value.	PASS

Testing Conclusion

During testing two significant bugs were identified and fixed. The first bug was in udp_scan and affected how the ports were scanned. A misplaced return statement inside the for loop caused a multi port scanner to stop after the first port. The second major bug was a missing continue in the get_scan_type function and how invalid inputs could’ve been accepted and return None rather than re prompting the user. With testing concluded the code can now be finalised with any issues or upgrades referred to in the future improvements section.

Conclusion

Mini Nmap has met six of its seven success criteria. The seventh however was only met partially. Although the TCP outputs align with Nmap the outputs diverge with firewall

filtered ports and UDP classified timeouts. Where Nmap identifies open|filtered and filtered, Mini Nmap reports a timeout state as the scanner cannot distinguish the different scenarios due to its simplicity. Even though Mini Nmap doesn’t match Nmap outputs entirely it has achieved the rest of its success criteria and met its original purpose serving both as a learning tool as well as a simplified version of Nmap.

Future Improvements

By considering the current state of the program there are various different improvements that can be implemented that would improve the program substantially. These improvements can range from functional additions such as new features or changes in existing code such as further development of UDP and TCP scans. Below is a list of all the proposed improvements:

- Advanced filtering and timeout TCP code.
- ICMP based filter detection.
- Improved UDP logic when handling ICMP responses.
- IPv6 Support.
- Host reachability checks.
- Stealth scanning techniques.
- OS fingerprinting
- Option to display results in file instead of just CLI.